

SKILL-4 K. Samanth

The screenshot shows the SQL Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with 'bankdb' selected. The main query window contains the following SQL statements:

```
1 * CREATE DATABASE IF NOT EXISTS BankDB;
2 * USE BankDB;
3 * DROP TABLE IF EXISTS Transaction_Audit;
4 * DROP TABLE IF EXISTS Bank_Transaction;
5 * DROP TABLE IF EXISTS Loan;
```

The bottom pane shows the 'Output' window with the following execution results:

#	Time	Action	Message	Duration / Fetch
1	12:36:23	CREATE DATABASE IF NOT EXISTS BankDB	1 row(s) affected. 1 warning(s): 1007 Can't create database 'bankdb'; database exists	0.016 sec
2	12:36:40	CREATE DATABASE IF NOT EXISTS BankDB	1 row(s) affected. 1 warning(s): 1007 Can't create database 'bankdb'; database exists	0.031 sec
3	12:36:40	USE BankDB	0 row(s) affected	0.000 sec
4	12:37:08	DROP TABLE IF EXISTS Transaction_Audit	0 row(s) affected. 1 warning(s): 1051 Unknown table 'bankdb.transaction_audit'	0.047 sec
5	12:37:18	DROP TABLE IF EXISTS Bank_Transaction	0 row(s) affected	0.094 sec
6	12:37:29	DROP TABLE IF EXISTS Loan	0 row(s) affected	0.094 sec

```
5 * DROP TABLE IF EXISTS Loan;
6 * DROP TABLE IF EXISTS Account;
7 * DROP TABLE IF EXISTS Customer;
```

✓	6	12:37:29	DROP TABLE IF EXISTS Loan	0 row(s) affected
✓	7	12:37:51	DROP TABLE IF EXISTS Account	0 row(s) affected
✓	8	12:38:01	DROP TABLE IF EXISTS Customer	0 row(s) affected

```

8  ● ○ CREATE TABLE Customer (
9      Customer_ID INT PRIMARY KEY,
10     Customer_Name VARCHAR(100) NOT NULL,
11     Phone VARCHAR(15),
12     Email VARCHAR(100),
13     City VARCHAR(50)
14 );
15  ● ○ CREATE TABLE Account (
16     Account_No INT PRIMARY KEY,
17     Customer_ID INT,
18     Account_Type VARCHAR(20),
19     Balance DECIMAL(12,2) DEFAULT 0,
20     Branch VARCHAR(50),
21     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
22 );

```

✓	9	12:38:33	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NU...	0 row(s) affected
✓	10	12:38:47	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected

```

23  ● ○ CREATE TABLE Bank_Transaction (
24     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
25     Account_No INT,
26     Transaction_Type VARCHAR(20),
27     Amount DECIMAL(12,2),
28     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
29     FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
30 );
31  ● ○ CREATE TABLE Loan (
32     Loan_ID INT PRIMARY KEY,
33     Customer_ID INT,
34     Loan_Type VARCHAR(30),
35     Loan_Amount DECIMAL(12,2),
36     Interest_Rate DECIMAL(5,2),
37     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
38 );

```

✓	11	12:39:15	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_...	0 row(s) affected
✓	12	12:39:29	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), ...	0 row(s) affected

```

39 • CREATE TABLE Transaction_Audit (
40     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
41     Transaction_ID INT,
42     Account_No INT,
43     Transaction_Type VARCHAR(20),
44     Amount DECIMAL(12,2),
45     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
46 );

```

13 12:39:57 CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID I... 0 row(s) affected

```

53 • INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES
54 (10001, 101, 'Savings', 50000.00, 'Hyderabad'),
55 (10002, 102, 'Savings', 75000.00, 'Vijayawada'),
56 (10003, 103, 'Current', 120000.00, 'Bangalore'),
57 (10004, 104, 'Savings', 45000.00, 'Chennai'),
58 (10005, 105, 'Current', 90000.00, 'Hyderabad');
59 • INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES
60 (10001, 'DEPOSIT', 10000.00),
61 (10002, 'DEPOSIT', 15000.00),
62 (10003, 'WITHDRAW', 20000.00),
63 (10004, 'DEPOSIT', 5000.00),
64 (10005, 'WITHDRAW', 10000.00);
65 • INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES
66 (501, 101, 'Home Loan', 500000.00, 7.50),
67 (502, 102, 'Education Loan', 100000.00, 6.50),
68 (503, 103, 'Car Loan', 80000.00, 8.20),
69 (504, 104, 'Personal Loan', 50000.00, 10.50);

```

Output :

#	Time	Action	Message
1	13:33:17	USE BankDB	0 row(s) affected
2	13:33:22	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'Sa...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
3	13:33:36	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
4	13:33:48	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, 'Ho...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

```

70 • USE BankDB;
71 DELIMITER //
72
73 • DROP PROCEDURE IF EXISTS GetAllCustomers //
74 • CREATE PROCEDURE GetAllCustomers()
75 BEGIN
76     SELECT * FROM Customer;
77 END //
78
79 • DROP PROCEDURE IF EXISTS GetAccountDetails //
80 • CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT)
81 BEGIN
82     SELECT * FROM Account WHERE Account_No = p_Account_No;
83 END //
84
85 • DROP PROCEDURE IF EXISTS GetCustomerAccounts //
86 • CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT)
87 BEGIN
88     SELECT C.Customer_ID, C.Customer_Name, A.Account_No, A.Account_Type, A.Balance, A.Branch

```

```

89     FROM Customer C
90     JOIN Account A ON C.Customer_ID = A.Customer_ID
91     WHERE C.Customer_ID = p_Customer_ID;
92 END //
93
94 • DROP PROCEDURE IF EXISTS DepositMoney //
95 • CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2))
96 BEGIN
97     UPDATE Account SET Balance = Balance + p_Amount WHERE Account_No = p_Account_No;
98 END //
99
100 • DROP PROCEDURE IF EXISTS WithdrawMoney //
101 • CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2))
102 BEGIN
103     UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No;
104 END //
105
106 • DROP PROCEDURE IF EXISTS TransferMoney //
107 • CREATE PROCEDURE TransferMoney(
108     IN SenderAccount INT,
109     IN ReceiverAccount INT,

```

```

110     IN TransferAmount DECIMAL(12,2)
111 )
112 BEGIN
113     DECLARE SenderBalance DECIMAL(12,2);
114     SELECT Balance INTO SenderBalance FROM Account WHERE Account_No = SenderAccount;
115
116     IF SenderBalance < TransferAmount THEN
117         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
118     ELSE
119         UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount;
120         UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount;
121     END IF;
122 END //
123
124 • DROP PROCEDURE IF EXISTS GetCustomerLoans //
125 • CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT)
126 BEGIN
127     SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate
128     FROM Customer C
129     JOIN Loan L ON C.Customer_ID = L.Customer_ID
130     WHERE C.Customer_ID = p_Customer_ID;

```

```

113 DECLARE SenderBalance DECIMAL(12,2);
114 SELECT Balance INTO SenderBalance FROM Account WHERE Account_No = SenderAccount;
115
116 IF SenderBalance < TransferAmount THEN
117     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
118 ELSE
119     UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount;
120     UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount;
121 END IF;
122 END //
123
124 DROP PROCEDURE IF EXISTS GetCustomerLoans //
125 CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT)
126 BEGIN
127     SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate
128     FROM Customer C
129     JOIN Loan L ON C.Customer_ID = L.Customer_ID
130     WHERE C.Customer_ID = p_Customer_ID;
131 END //
132
133 DROP PROCEDURE IF EXISTS HighBalanceAccounts //

```

```

134 CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2))
135 BEGIN
136     SELECT * FROM Account WHERE Balance >= MinimumBalance ORDER BY Balance DESC;
137 END //
138
139 DROP PROCEDURE IF EXISTS GetBalance //
140 CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2))
141 BEGIN
142     SELECT Balance INTO p_Balance FROM Account WHERE Account_No = p_Account_No;
143 END //
144
145 DELIMITER ;

```

✓	14	13:43:26	USE BankDB	0 row(s) affected
✓	15	13:43:26	DROP PROCEDURE IF EXISTS GetAllCustomers	0 row(s) affected
✓	16	13:43:26	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
⚠	17	13:43:26	DROP PROCEDURE IF EXISTS GetAccountDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetAccountDetails does not exist
✓	18	13:43:26	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	19	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerAccounts does not exist
⚠	19	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerAccounts does not exist
✓	20	13:43:26	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate FROM Customer C JOIN Loan L ON C.Customer_ID = L.Customer_ID WHERE C.Customer_ID = p_Customer_ID; END	0 row(s) affected
⚠	21	13:43:26	DROP PROCEDURE IF EXISTS DepositMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.DepositMoney does not exist
✓	22	13:43:26	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance + p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	23	13:43:26	DROP PROCEDURE IF EXISTS WithdrawMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.WithdrawMoney does not exist
✓	24	13:43:26	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	23	13:43:26	DROP PROCEDURE IF EXISTS WithdrawMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.WithdrawMoney does not exist
✓	24	13:43:26	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	25	13:43:26	DROP PROCEDURE IF EXISTS TransferMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.TransferMoney does not exist
✓	26	13:43:26	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN TransferAmount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount; UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount; END	0 row(s) affected
⚠	27	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerLoans	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerLoans does not exist
✓	28	13:43:26	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate FROM Customer C JOIN Loan L ON C.Customer_ID = L.Customer_ID WHERE C.Customer_ID = p_Customer_ID; END	0 row(s) affected

#	Time	Action	Message
27	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerLoans	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerLoans does not exist
28	13:43:26	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN SELECT C.Customer_Name, L.L...	0 row(s) affected
29	13:43:26	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.HighBalanceAccounts does not exist
30	13:43:26	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * FRO...	0 row(s) affected
31	13:43:26	DROP PROCEDURE IF EXISTS GetBalance	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetBalance does not exist
32	13:43:26	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN SELE...	0 row(s) affected

```

149 • DROP TRIGGER IF EXISTS CheckBalance //
150 • CREATE TRIGGER CheckBalance
151 BEFORE UPDATE ON Account
152 FOR EACH ROW
153 BEGIN
154 IF NEW.Balance < 0 THEN
155     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
156 END IF;
157 END //
158
159 • DROP TRIGGER IF EXISTS CheckTransactionAmount //
160 • CREATE TRIGGER CheckTransactionAmount
161 BEFORE INSERT ON Bank_Transaction

```

```

162 FOR EACH ROW
163 BEGIN
164 IF NEW.Amount <= 0 THEN
165     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
166 END IF;
167 END //
168
169 • DROP TRIGGER IF EXISTS TransactionAudit //
170 • CREATE TRIGGER TransactionAudit
171 AFTER INSERT ON Bank_Transaction
172 FOR EACH ROW
173 BEGIN
174     INSERT INTO Transaction_Audit (Transaction_ID, Account_No, Transaction_Type, Amount)
175     VALUES (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
176 END //
177
178 • DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction //
179 • CREATE TRIGGER UpdateBalanceAfterTransaction
180 AFTER INSERT ON Bank_Transaction
181 FOR EACH ROW
182 BEGIN

```

```

195 DECLARE CurrentBalance DECIMAL(12,2);
196 SELECT Balance INTO CurrentBalance FROM Account WHERE Account_No = NEW.Account_No;
197
198 IF NEW.Transaction_Type = 'WITHDRAW' AND NEW.Amount > CurrentBalance THEN
199     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
200 END IF;
201 END //
202
203 DELIMITER ;

```

⚠	34	13:46:04	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	35	13:46:04	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bala...	0 row(s) affected
⚠	36	13:46:04	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	37	13:46:04	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BE...	0 row(s) affected
⚠	38	13:46:04	DROP TRIGGER IF EXISTS TransactionAudit	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist

✅	39	13:46:04	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS...	0 row(s) affected
⚠	40	13:46:04	DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	41	13:46:04	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW...	0 row(s) affected
⚠	42	13:46:04	DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist

⚠	38	13:46:04	DROP TRIGGER IF EXISTS TransactionAudit	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	39	13:46:04	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS...	0 row(s) affected
⚠	40	13:46:04	DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	41	13:46:04	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW...	0 row(s) affected
⚠	42	13:46:04	DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	43	13:46:04	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH ROW ...	0 row(s) affected

```

204 • USE BankDB;
205 DELIMITER //
206
207 • DROP TRIGGER IF EXISTS CheckBalance //
208 • CREATE TRIGGER CheckBalance
209 BEFORE UPDATE ON Account
210 FOR EACH ROW
211 BEGIN
212     IF NEW.Balance < 0 THEN
213         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
214     END IF;
215 END //
216
217 • DROP TRIGGER IF EXISTS CheckTransactionAmount //
218 • CREATE TRIGGER CheckTransactionAmount
219 BEFORE INSERT ON Bank_Transaction

```

✅	45	13:48:06	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
✅	46	13:48:06	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bala...	0 row(s) affected
✅	47	13:48:06	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
✅	48	13:48:06	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BE...	0 row(s) affected

```

220     FOR EACH ROW
221     BEGIN
222     IF NEW.Amount <= 0 THEN
223         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
224     END IF;
225     END //
226
227 • DROP TRIGGER IF EXISTS TransactionAudit //
228 • CREATE TRIGGER TransactionAudit
229     AFTER INSERT ON Bank_Transaction
230     FOR EACH ROW
231     BEGIN
232         INSERT INTO Transaction_Audit (Transaction_ID, Account_No, Transaction_Type, Amount)
233         VALUES (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
234     END //
235
236 • DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction //
237 • CREATE TRIGGER UpdateBalanceAfterTransaction
238     AFTER INSERT ON Bank_Transaction
239     FOR EACH ROW
240     BEGIN

```

```

✓ 49 13:48:06 DROP TRIGGER IF EXISTS TransactionAudit 0 row(s) affected
✓ 50 13:48:06 CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS... 0 row(s) affected
✓ 51 13:48:06 DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction 0 row(s) affected
✓ 52 13:48:06 CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW... 0 row(s) affected
✓ 53 13:48:06 DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal 0 row(s) affected
✓ 54 13:48:06 CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH ROW 0 row(s) affected

```

```

253     DECLARE CurrentBalance DECIMAL(12,2);
254     SELECT Balance INTO CurrentBalance FROM Account WHERE Account_No = NEW.Account_No;
255
256     IF NEW.Transaction_Type = 'WITHDRAW' AND NEW.Amount > CurrentBalance THEN
257         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
258     END IF;
259     END //
260
261 DELIMITER ;

```

```

265 • CALL GetAllCustomers();
266 • CALL GetAccountDetails(10001);

```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content: IA	
	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

```

266 • CALL GetAccountDetails(10001);
267 • CALL GetCustomerAccounts(101);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad

```

267 • CALL GetCustomerAccounts(101);
268 • CALL HighBalanceAccounts(50000);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: I A

	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

```

268 • CALL HighBalanceAccounts(50000);
269

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10001	101	Savings	50000.00	Hyderabad

```

271 • SELECT * FROM Transaction_Audit;

```

Result Grid | Filter Rows: | Edit: | Export/Import:

	Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
▶	NULL	NULL	NULL	NULL	NULL	NULL